Back To Basics
Templates

LAURENT CARLIER



Cppcon
The C++ Conference

20
26



September 12 - 18

Slides best viewed online at https://www.laurentcarlier.com/b2b-template-cppcon-2026/

GOALS OF THIS PRESENTATION

GOALS OF THIS PRESENTATION

TAKE HOME SOMETHING YOU CAN USE

GOALS OF THIS PRESENTATION

TAKE HOME SOMETHING YOU CAN USE

- Template enables type safety

GOALS OF THIS PRESENTATION

TAKE HOME SOMETHING YOU CAN USE

- Template enables type safety
- Template is performant

GOALS OF THIS PRESENTATION

TAKE HOME SOMETHING YOU CAN USE

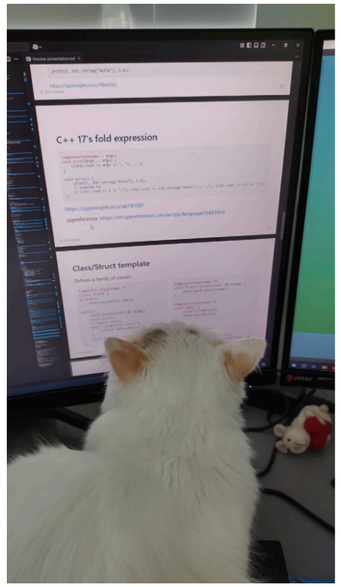
- Template enables type safety
- Template is performant
- Template enable generic programming

GOALS OF THIS PRESENTATION

TEMPLATE DOESN'T NEED TO BE COMPLICATED

GOALS OF THIS PRESENTATION

TEMPLATE DOESN'T NEED TO BE COMPLICATED



PRESENTATION PHILOSOPHY

PRESENTATION PHILOSOPHY

- Teach by examples

PRESENTATION PHILOSOPHY

- Teach by examples
- Explain the feature used in the examples

PRESENTATION PHILOSOPHY

- Teach by examples
- Explain the feature used in the examples
- Warn about common pitfalls

PRESEN

- Teach by examp
- Explain the feat
- Warn about com

Nicolai Josuttis, Back To Basics: Templates, CppCon 2022



https://youtu.be/HqsEHG0QJXU

⏰ AGENDA OF TODAY ⏰

- Syntax & function templates
- Implicit requirements
- Template argument deduction
- Variadic templates & perfect forwarding
- Class templates
- 3 production code examples

TEMPLATE SYNTAX

A template defines a family of **functions**, **classes**, **type aliases**, **variables** or **concepts**.

```
template< [parameter-list] >
recipe
```

TEMPLATE SYNTAX

A template defines a family of **functions**, **classes**, **type aliases**, **variables** or **concepts**.

```
template< [parameter-list] >
recipe
```

`parameter-list` can be:

- type parameter e.g. `std::string`
- non type parameter (aka NTTP) e.g. enum value, constexpr value
- parameter pack using ellipsis `...`
- template parameter e.g. `template<typename> class`

TEMPLATE SYNTAX

A template defines a family of **functions**, **classes**, **type aliases**, **variables** or **concepts**.

```
template< [parameter-list] >
recipe
```

`parameter-list` can be:

- type parameter e.g. `std::string`
- non type parameter (aka NTTP) e.g. enum value, constexpr value
- parameter pack using ellipsis `...`
- template parameter e.g. `template<typename> class`

`recipe` can be:

- function definition
- a class/struct definition
- a type alias definition
- variable definition
- concept definition

FUNCTION TEMPLATE

DEFINES A FAMILY OF FUNCTIONS

```
1 template<typename Swappable>
2 void mySwap(Swappable& a, Swappable& b) {
3   Swappable tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
```

FUNCTION TEMPLATE

DEFINES A FAMILY OF FUNCTIONS

```
1 template<typename Swappable>
2 void mySwap(Swappable& a, Swappable& b) {
3   Swappable tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
```

💡 Give good name to template parameters

FUNCTION TEMPLATE

DEFINES A FAMILY OF FUNCTIONS

```
1 template<typename Swappable>
2 void mySwap(Swappable& a, Swappable& b) {
3   Swappable tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
```

💡 Give good name to template parameters

⚠️ The compiler does not produce any code if the template is not instantiated

FUNCTION TEMPLATE

DEFINES A FAMILY OF FUNCTIONS

```
template<typename Swappable>
void mySwap(Swappable& a, Swappable& b) {
  Swappable tmp = std::move(a);
  a = std::move(b);
  b = std::move(tmp);
}
```

```
1 int a = 5, b = 10;
2 mySwap(a, b);
3 assert(a == 10 && b == 5);
```

FUNCTION TEMPLATE

DEFINES A FAMILY OF FUNCTIONS

```
template<typename Swappable>
void mySwap(Swappable& a, Swappable& b) {
  Swappable tmp = std::move(a);
  a = std::move(b);
  b = std::move(tmp);
}
```

```
1 int a = 5, b = 10;
2 mySwap(a, b);
3 assert(a == 10 && b == 5);
```

```
1 float c = 3.14f, d = 2.71f;
2 mySwap(c, d);
3 assert(c == 2.71f && d == 3.14f);
```

FUNCTION TEMPLATE

DEFINES A FAMILY OF FUNCTIONS

```
template<typename Swappable>
void mySwap(Swappable& a, Swappable& b) {
  Swappable tmp = std::move(a);
  a = std::move(b);
  b = std::move(tmp);
}
```

```
1 int a = 5, b = 10;
2 mySwap(a, b);
3 assert(a == 10 && b == 5);
```

```
1 float c = 3.14f, d = 2.71f;
2 mySwap(c, d);
3 assert(c == 2.71f && d == 3.14f);
```

⚠️ Calling the function with 2 different types fails

```
1 float f = 3.14f;
2 int i = 42;
3 mySwap(i, f); // ❌ Does not compile
```

FUNCTION TEMPLATE

DEFINES A FAMILY OF FUNCTIONS 🔎

cppinsights.io/s/b05df164

Source:

```
1  template<typename Swappable>
2  void mySwap(Swappable& a, Swappable& b) {
3    Swappable tmp = std::move(a);
4    a = std::move(b);
5    b = std::move(tmp);
6  }
7
8  int a = 5, b = 10;
9  mySwap(a, b);
10
11 float c = 3.14f, d = 2.71f;
12 mySwap(c, d);
```

Insight:

```
1  template<>
2  void mySwap<int>(int& a, int& b)
3  {
4    int tmp = std::move(a);
5    a = std::move(b);
6    b = std::move(tmp);
7  }
8
9  template<>
10 void mySwap<float>(float& a, float& b)
11 {
12   float tmp = std::move(a);
13   a = std::move(b);
14   b = std::move(tmp);
15 }
```

⚠️ The compiler generates code for each instantiation.

https://cppinsights.io/s/b05df164

FUNCTION TEMPLATE

HOW ARE FUNCTION TEMPLATES COMPILED?

Two-phase name lookup

```
1 template<typename Swappable>
2 void mySwap(Swappable& a, Swappable& b) {
3   Swappable tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
7
8 int a = 5, b = 10;
9 mySwap(a, b);
```

FUNCTION TEMPLATE

HOW ARE FUNCTION TEMPLATES COMPILED?

Two-phase name lookup

```
1 template<typename Swappable>
2 void mySwap(Swappable& a, Swappable& b) {
3   Swappable tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
7
8 int a = 5, b = 10;
9 mySwap(a, b);
```

First, the compiler checks for syntax and resolve non-dependent names identifier.

FUNCTION TEMPLATE

HOW ARE FUNCTION TEMPLATES COMPILED?

Two-phase name lookup

```
1 template<typename Swappable>
2 void mySwap(Swappable& a, Swappable& b) {
3   Swappable tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
7
8 int a = 5, b = 10;
9 mySwap(a, b);
```

First, the compiler checks for syntax and resolve non-dependent names identifier.

Second the template argument is substituted in for the template parameter

FUNCTION TEMPLATE

HOW ARE FUNCTION TEMPLATES COMPILED?

Two-phase name lookup

```
1 template<typename Swappable>
2 void mySwap(Swappable& a, Swappable& b) {
3   Swappable tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
7
8 int a = 5, b = 10;
9 mySwap(a, b);
```

First, the compiler checks for syntax and resolve non-dependent names identifier.

Second the template argument is substituted in for the template parameter

⚠️ The code can compile until a function template is called

FUNCTION TEMPLATE

IMPLICIT REQUIREMENTS

Implicit requirements are the sets of operations and properties that a type must support for the template to compile

FUNCTION TEMPLATE

IMPLICIT REQUIREMENTS

Implicit requirements are the sets of operations and properties that a type must support for the template to compile

```
1 template<typename T>
2 void mySwap(T& a, T& b) {
3   T tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
```

What are the implicit requirements for the type `T`?



FUNCTION TEMPLATE

IMPLICIT REQUIREMENTS

Implicit requirements are the sets of operations and properties that a type must support for the template to compile

```
1 template<typename T>
2 void mySwap(T& a, T& b) {
3   T tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
```

What are the implicit requirements for the type `T`?

🙋‍♂️🙋‍♀️

- Must be move constructible i.e have a valid `T(T&&)` constructor
- Must be move assignable i.e have a valid `T& operator=(T&&)` assignment operator

FUNCTION TEMPLATE

IMPLICIT REQUIREMENTS

Implicit requirements are the sets of operations and properties that a type must support for the template to compile

```
1 template<typename T>
2 void mySwap(T& a, T& b) {
3   T tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
```

What are the implicit requirements for the type `T`?

🙋‍♂️🙋

- Must be move constructible i.e have a valid `T(T&&)` constructor
- Must be move assignable i.e have a valid `T& operator=(T&&)` assignment operator

💡 C++ 20 formally defines the requirement via the `swappable` concept.

💡 Before C++20, document the requirement explicitly using comments.

FUNCTION TEMPLATE

IMPLICIT REQUIREMENTS

Implicit requirements ar[illegible]ort for the template to compile

```
1 template<typename T>
2 void mySwap(T& a, T& b) {
3   T tmp = std::move(a);
4   a = std::move(b);
5   b = std::move(tmp);
6 }
```

What are the implicit rec[illegible]

🙋‍♂️🙋‍♀️

- Must be move constructible i.e have a valid `T(T&&)` constructor
- Must be move assignable i.e have a valid `T& operator=(T&&)` assignment operator

💡 C++ 20 formally defines the requirement via the `swappable` concept.

💡 Before C++20, document the requirement explicitly using comments.

ℹ️ There are numerous possible implicit requirements

For instance:

- `T` must have a public class member variable `T.data`
- `T` must have a function member `T::fun(int)`
- `T` must have a nested type `T::TYPE`
- `T` must have a `static` function member

👉 Implicit requirement is a very important notion 👈

TEMPLATE ARGUMENT DEDUCTION

GENERAL

Templates argument are deduced from the function arguments

TEMPLATE ARGUMENT DEDUCTION

GENERAL

Templates argument are deduced from the function arguments

For instance

```
1  template<typename T>
2  T myMax(T a, T b) {
3    return a > b ? a : b;
4  }
5
6  void fun() {
7    int a = 5, b = 4;
8    auto c = myMax(a, b);
9    static_assert(std::is_same_v<decltype(c), int>);
10 }
```

TEMPLATE ARGUMENT DEDUCTION

GENERAL

Templates argument are deduced from the function arguments

For instance

```
1  template<typename T>
2  T myMax(T a, T b) {
3    return a > b ? a : b;
4  }
5
6  void fun() {
7    int a = 5, b = 4;
8    auto c = myMax(a, b);
9    static_assert(std::is_same_v<decltype(c), int>);
10 }
```

Calling `myMax(a, b)` deduces `T = int`

TEMPLATE ARGUMENT DEDUCTION

TYPE DEDUCTION

Another example

```
1 template<typename T>
2 T myMax(T a, T b) {
3   return a > b ? a : b;
4 }
5
6 void fun() {
7   auto c = myMax(5.1, 5.2);
8 }
```

What does `T` deduced to ?

🙋‍♂️🤷‍♀️

TEMPLATE ARGUMENT DEDUCTION

TYPE DEDUCTION

Another example

```
1 template<typename T>
2 T myMax(T a, T b) {
3   return a > b ? a : b;
4 }
5
6 void fun() {
7   auto c = myMax(5.1, 5.2);
8 }
```

What does `T` deduced to ?

🙋‍♂️🤷‍♀️

```
double
```

TEMPLATE ARGUMENT DEDUCTION

EXPLICIT TEMPLATE ARGUMENTS

Provide the template argument **explicitly** to override deduction

```
1 template<typename T>
2 T myMax(T a, T b) {
3   return a > b ? a : b;
4 }
5
6 void fun() {
7   auto c = myMax<float>(5.1, 5.2);
8 }
```

TEMPLATE ARGUMENT DEDUCTION

EXPLICIT TEMPLATE ARGUMENTS

Provide the template argument **explicitly** to override deduction

```
1 template<typename T>
2 T myMax(T a, T b) {
3   return a > b ? a : b;
4 }
5
6 void fun() {
7   auto c = myMax<float>(5.1, 5.2);
8 }
```

⚠️ Template arguments are never deduced from the return type ⚠️

```
template<typename T>
T parse(const std::string& text);

int i  = parse("42");       // ❌ Does not compile
auto i = parse<int>("42"); // 👍
```

TEMPLATE ARGUMENT DEDUCTION

HOW `cv qualifiers` ARE DEDUCED

One question decides everything: is the parameter a **reference** or not?

```
1 template<typename T>
2 void funcValue(T a);
3
4 template<typename T>
5 void funcRefParam(T& a);
```

```
1 int x = 5;
2 const int cx = 5;
3
4 funcValue(x);  // T is deduced as int
5 funcValue(cx); // T is deduced as int (const is dropped)
6
7 funcRefParam(x);  // T is deduced as int
8 funcRefParam(cx); // T is deduced as const int (const is kept)
```

TEMPLATE ARGUMENT DEDUCTION

HOW `cv qualifiers` ARE DEDUCED

One question decides everything: is the parameter a **reference** or not?

```
1 template<typename T>
2 void funcValue(T a);
3
4 template<typename T>
5 void funcRefParam(T& a);
```

```
1 int x = 5;
2 const int cx = 5;
3
4 funcValue(x);  // T is deduced as int
5 funcValue(cx); // T is deduced as int (const is dropped)
6
7 funcRefParam(x);  // T is deduced as int
8 funcRefParam(cx); // T is deduced as const int (const is kept)
```

⚠️ A `const T&` parameter drops it again

```
1 template<typename T>
2 void funcConstRefParam(const T& a);
3
4 funcConstRefParam(cx); // T is int, but a is still const int&
```

TEMPLATE ARGUMENT DEDUCTION

HOW `cv qualifiers` ARE DEDUCED

One question decides everything: is the parameter a **reference** or not?

```
1 template<typename T>
2 void funcValue(T a);
3
4 template<typename T>
5 void funcRefParam(T& a);
```

> 💡 **Rule of thumbs** 💡
>
> - By value → cv-qualifiers are **dropped**
> - By reference → cv-qualifiers are **preserved**

```
uced as int
uced as int (const is dropped)

deduced as int
6 funcRefParam(cx); // T is deduced as const int (const is kept)
```

⚠️ A `const T&` parameter drops it again

```
1 template<typename T>
2 void funcConstRefParam(const T& a);
3
4 funcConstRefParam(cx); // T is int, but a is still const int&
```

TEMPLATE ARGUMENT DEDUCTION

ARRAYS AND POINTERS

```
1 template<typename T>
2 void funcValue(T arg);
3
4 template<typename T>
5 void funcRefParam(T& arg);
```

```
1  int arr[5];
2  int* ptrArr = arr;
3
4  // the pointer remains a pointer
5  funcValue(ptrArr); // T is deduced as int*
6
7  // the reference keeps the int* type
8  funcRefParam(ptrArr); // T is deduced as int*. But arg is now int*& (dangerous)
9
10 // the array decays to a pointer, the size is lost
11 funcValue(arr);    // T is deduced as int*
12
13 // the reference keeps the array type
14 funcRefParam(arr); // T is deduced as int[5]
```

TEMPLATE ARGUMENT DEDUCTION

ARRAYS AND POINTERS

```cpp
template<typename T>
void funcValue(T arg);

template<typename T>
void funcRefParam(T& arg);
```

```cpp
int arr[5];
int* ptrArr = arr;

// the pointer remains a pointer
funcValue(ptrArr); // T is deduced as int*

// the reference keeps the int* type
funcRefParam(ptrArr); // T is deduced as int*. But arg is now int*& (dangerous)

// the array decays to a pointer, the size is lost
funcValue(arr);    // T is deduced as int*

// the reference keeps the array type
funcRefParam(arr); // T is deduced as int[5]
```

💡 A reference parameter can even deduce the size as an NTTP 💡

```cpp
template<typename T, std::size_t N>
constexpr std::size_t arraySize(T(&)[N]) { return N; }

int arr[5];
static_assert(arraySize(arr) == 5);
```

TEMPLATE ARGUMENT DEDUCTION

⚠️ WHY `T&` IS DANGEROUS WITH POINTERS ⚠️

A reference parameter will happily deduce `T` as a **pointer** type.

```
1  template<typename T>
2  void dangerous(T& retry) {
3    // ... some logic
4    ++retry;
5  }
6
7  int retries = 0;
8  int* retriesPtr = &retries;
9
10 // 😱 Forgot the '*' 😱
11 dangerous(retriesPtr); // T is int*, so retry is int*&
12 // retriesPtr is now pointing to an invalid memory address
13 printf("%d\n", *retriesPtr);   // 💥 disaster 💥
```

TEMPLATE ARGUMENT DEDUCTION

⚠️ WHY `T&` IS DANGEROUS WITH POINTERS ⚠️

A reference parameter will happily deduce `T` as a **pointer** type.

```
template<typename T>
void dangerous(T& retry) {
  // ... some logic
  ++retry;
}

int retries = 0;
int* retriesPtr = &retries;

// 😱 Forgot the '*' 😱
dangerous(retriesPtr); // T is int*, so retry is int*&
// retriesPtr is now pointing to an invalid memory address
printf("%d\n", *retriesPtr);   // 💥 disaster 💥
```

TEMPLATE ARGUMENT DEDUCTION

⚠️ WHY `T&` IS DANGEROUS WITH POINTERS ⚠️

A reference parameter will happily deduce `T` as a **pointer** type.

```
template<typename T>
void dangerous(T& retry) {
  // ... some logic
  ++retry;
}

int retries = 0;
int* retriesPtr = &retries;

// 😱 Forgot the '*' 😱
dangerous(retriesPtr); // T is int*, so retry is int*&
// retriesPtr is now pointing to an invalid memory address
printf("%d\n", *retriesPtr);   // 💥 disaster 💥
```

TEMPLATE ARGUMENT DEDUCTION

⚠️ WHY `T&` IS DANGEROUS WITH POINTERS ⚠️

A reference parameter will happily deduce `T` as a **pointer** type.

```
template<typename T>
void dangerous(T& retry) {
  // ... some logic
  ++retry;
}

int retries = 0;
int* retriesPtr = &retries;

// 😱 Forgot the '*' 😱
dangerous(retriesPtr); // T is int*, so retry is int*&
// retriesPtr is now pointing to an invalid memory address
printf("%d\n", *retriesPtr);   // 💥 disaster 💥
```

The **pointer** is incremented, not retries

TEMPLATE ARGUMENT DEDUCTION

⚠️ WHY `T&` IS DANGEROUS WITH POINTERS ⚠️

A reference parameter will happily deduce `T` as a **pointer** type.

```
1  template<typename T>
2  void dangerous(T& retry) {
3    // ... some logic
4    ++retry;
5  }
6
7  int retries = 0;
8  int* retriesPtr = &retries;
9
10 // 😱 Forgot the '*' 😱
11 dangerous(retriesPtr); // T is int*, so retry is int*&
12 // retriesPtr is now pointing to an invalid memory address
13 printf("%d\n", *retriesPtr);   // 💥 disaster 💥
```

The **pointer** is incremented, not retries

- ⚠️ Think twice before using a non-const reference parameter

TEMPLATE ARGUMENT DEDUCTION

⚠️ WHY `T&` IS DANGEROUS WITH POINTERS ⚠️

A reference parameter will happily deduce `T` as a **pointer** type.

```
1  template<typename T>
2  void dangerous(T& retry) {
3    // ... some logic
4    ++retry;
5  }
6
7  int retries = 0;
8  int* retriesPtr = &retries;
9
10 // 😱 Forgot the '*' 😱
11 dangerous(retriesPtr); // T is int*, so retry is int*&
12 // retriesPtr is now pointing to an invalid memory address
13 printf("%d\n", *retriesPtr);   // 💥 disaster 💥
```

The **pointer** is incremented, not retries

- ⚠️ Think twice before using a non-const reference parameter
- 💡 Validate the deduced type with `static_assert` and `<type_traits>`, e.g. `std::is_pointer_v`, `std::is_const_v`

TEMPLATE ARGUMENT DEDUCTION

⚠️ WHY `T&` IS DANGEROUS WITH POINTERS ⚠️

A reference parameter will happily deduce `T` as a **pointer** type.

```cpp
template<typename T>
void dangerous(T& retry) {
  // ... some logic
  ++retry;
}

int retries = 0;
int* retriesPtr = &retries;

// 😱 Forgot the '*' 😱
dangerous(retriesPtr); // T is int*, so retry is int*&
// retriesPtr is now pointing to an invalid memory address
printf("%d\n", *retriesPtr);   // 💥 disaster 💥
```

The **pointer** is incremented, not retries

- ⚠️ Think twice before using a non-const reference parameter
- 💡 Validate the deduced type with `static_assert` and <type_traits>, e.g. `std::is_pointer_v`, `std::is_const_v`
- 💡 From C++20, use concepts

WHEN THE DEDUCED TYPE SURPRISES YOU

A GENERIC `myMax`

```
1 template<typename T>
2 T myMax(T a, T b) {
3   return a > b ? a : b;
4 }
```

What does the following call return? 🤔

```
myMax("hello", "world");
```

🤷‍♂️🤷‍♀️

WHEN THE DEDUCED TYPE SURPRISES YOU

A GENERIC `myMax`

```
1 template<typename T>
2 T myMax(T a, T b) {
3   return a > b ? a : b;
4 }
```

What does the following call return? 🤔

```
myMax("hello", "world");
```

🤷‍♂️🤷‍♀️

We don't know the result 🤷‍♂️. Pointer comparison

The instantiated function is

```
1 const char* myMax(const char* a, const char* b) {
2   return a > b ? a : b;
```

WHEN THE DEDUCED TYPE SURPRISES YOU

FIX: PROVIDE A FUNCTION OVERLOAD

A non-templated overload function gives a custom implementation for a specific type.

```
1 const char* myMax(const char* a, const char* b) {
2   return std::strcmp(a, b) > 0 ? a : b;
3 }
```

Now `myMax("hello", "world")` returns the pointer to `"world"` 👍

WHEN THE DEDUCED TYPE SURPRISES YOU

FIX: PROVIDE A FUNCTION OVERLOAD

A non-templated overload function gives a custom implementation for a specific type.

```
1 const char* myMax(const char* a, const char* b) {
2   return std::strcmp(a, b) > 0 ? a : b;
3 }
```

Now `myMax("hello", "world")` returns the pointer to `"world"` 👍

💡 Out of reach for a macro: the pre-processor cannot overload 💡

PUTTING TEMPLATES TO WORK

GENERIC PROGRAMMING

Iterate over any kind of iterables

```
1 template<typename Iterable>
2 void print(Iterable& c) {
3   for (const auto& element : c) {
4     std::cout << element << "\n";
5   }
6 }
```

PUTTING TEMPLATES TO WORK

GENERIC PROGRAMMING

Iterate over any kind of iterables

```
template<typename Iterable>
void print(Iterable& c) {
  for (const auto& element : c) {
    std::cout << element << "\n";
  }
}
```

Works with

Any standard containers (vector, list, set, map, ...)

```
std::vector<int> v = {1, 2, 3};
print(v);
```

C++ 20's span and ranges

```
std::span<int> s = {v};
print(s);

std::vector<int> v = {1, 2, 3, 4, 5};
auto even = v | std::views::filter([](int n) { return n % 2 == 0; });

print(even);
```

C-style arrays

```
int arr[] = {1, 2, 3};
print(arr);
```

PUTTING TEMPLATES TO WORK

GENERIC PROGRAMMING

Iterate over any kind of iterables

```
1 template<typename Iterable>
2 void print(Iterable& c) {
3   for (const auto& element : c) {
4     std::cout << element << "\n";
5   }
6 }
```

Works with

Custom types

```
1 struct Graph { ... };
2 // Make Graph printable by adding an overload for print
3 void print(Graph& g) { ... }
4
5 Graph g;
6 print(g); // 👍
```

Extensible without ever touching `print` 👍

PUTTING TEMPLATES TO WORK

"POLYMORPHISM" WITHOUT INHERITANCE

Polymorphic behaviour between unrelated types

```
 1 template<typename T>
 2 constexpr int legs(T& ll) { return ll.legs(); }
 3
 4 struct Cow {  // not in any class hierarchy
 5   constexpr int legs() { return 4; }
 6 };
 7
 8 struct Duck { // not in any class hierarchy
 9   constexpr int legs() { return 2; }
10 };
11
12 int main()
13 {
14   Cow c;
15   Duck d;
```

Example from Bjarne Stroustrup (apologies for the artificial example)

PUTTING TEMPLATES TO WORK

"POLYMORPHISM" WITHOUT INHERITANCE

Polymorphic behaviour between unrelated types

```
 1 template<typename T>
 2 constexpr int legs(T& ll) { return ll.legs(); }
 3
 4 struct Cow {  // not in any class hierarchy
 5   constexpr int legs() { return 4; }
 6 };
 7
 8 struct Duck { // not in any class hierarchy
 9   constexpr int legs() { return 2; }
10 };
11
12 int main()
13 {
14   Cow c;
15   Duck d;
```

Example from Bjarne Stroustrup (apologies for the artificial example)

PUTTING TEMPLATES TO WORK

"POLYMORPHISM" WITHOUT INHERITANCE

Polymorphic behaviour between unrelated types

```
 1 template<typename T>
 2 constexpr int legs(T& ll) { return ll.legs(); }
 3
 4 struct Cow {  // not in any class hierarchy
 5   constexpr int legs() { return 4; }
 6 };
 7
 8 struct Duck { // not in any class hierarchy
 9   constexpr int legs() { return 2; }
10 };
11
12 int main()
13 {
14   Cow c;
15   Duck d;
```

Example from Bjarne Stroustrup (apologies for the artificial example)

PUTTING TEMPLATES TO WORK

"POLYMORPHISM" WITHOUT INHERITANCE

Polymorphic behaviour between unrelated types

```
 6 };
 7
 8 struct Duck { // not in any class hierarchy
 9   constexpr int legs() { return 2; }
10 };
11
12 int main()
13 {
14   Cow c;
15   Duck d;
16
17   // resolved at compile time, no vtable, no common base
18   static_assert(legs(c) == 4);
19   static_assert(legs(d) == 2);
20 }
```

Example from Bjarne Stroustrup (apologies for the artificial example)

VARIADIC TEMPLATES

A VARIABLE NUMBER OF TEMPLATE PARAMETERS

VARIADIC TEMPLATES

A VARIABLE NUMBER OF TEMPLATE PARAMETERS

The ellipsis `...` says which parameter is a **pack**, and where it expands.

VARIADIC TEMPLATES

A VARIABLE NUMBER OF TEMPLATE PARAMETERS

The ellipsis `...` says which parameter is a **pack**, and where it expands.

```
template<typename T, typename ...Ts>
T sum_all(Ts... args) {
    std::array<T, sizeof...(Ts)> arr = { args... }; // pack expansion fills the array
    return std::accumulate(arr.begin(), arr.end(), 0);
}
```

VARIADIC TEMPLATES

A VARIABLE NUMBER OF TEMPLATE PARAMETERS

The ellipsis `...` says which parameter is a **pack**, and where it expands.

```cpp
template<typename T, typename ...Ts>
T sum_all(Ts... args) {
    std::array<T, sizeof...(Ts)> arr = { args... }; // pack expansion fills the array
    return std::accumulate(arr.begin(), arr.end(), 0);
}
```

```cpp
// Any number of arguments can be given to sum_all
auto total = sum_all<int>(1, 2); // total == 3
```

💡 `T` doesn't appears in any function parameter → it can only be given explicitly 💡

VARIADIC TEMPLATES

SUM_ALL UNDER THE HOOD 🔎

What the pack expansion really generates

cppinsights.io/s/b7e622a8

Source:

```
1 template<typename T, typename ...Ts>
2 T sum_all(Ts... args) {
3   std::array<T, sizeof...(Ts)> arr = {args...};
4   return std::accumulate(arr.begin(), arr.end(), 0);
5 }
6
7 auto total = sum_all<int>(1, 2);
```

Insight:

```
1 template<>
2 int sum_all<int, int, int>(int __args0, int __args1) {
3   std::array<int, 2> arr = {{__args0, __args1}};
4   return std::accumulate(arr.begin(), arr.end(), 0);
5 }
6
7 auto total = sum_all<int>(1, 2);
```

https://cppinsights.io/s/b7e622a8

VARIADIC TEMPLATES

SUM_ALL UNDER THE HOOD 🔎

What the pack expansion really generates

cppinsights.io/s/b7e622a8

Source:

```
1 template<typename T, typename ...Ts>
2 T sum_all(Ts... args) {
3   std::array<T, sizeof...(Ts)> arr = {args...};
4   return std::accumulate(arr.begin(), arr.end(), 0);
5 }
6
7 auto total = sum_all<int>(1, 2);
```

Insight:

```
1 template<>
2 int sum_all<int, int, int>(int __args0, int __args1) {
3   std::array<int, 2> arr = {{__args0, __args1}};
4   return std::accumulate(arr.begin(), arr.end(), 0);
5 }
6
7 auto total = sum_all<int>(1, 2);
```

https://cppinsights.io/s/b7e622a8

⚠️ If sum_all is called with 3 arguments a new version is generated

VARIADIC TEMPLATES

C++17'S FOLD EXPRESSIONS

Syntax

```
(pattern op ...);
```

VARIADIC TEMPLATES

C++17'S FOLD EXPRESSIONS

Syntax

```
(pattern op ...);
```

A fold repeats one expression for each element of the pack.

```
1 template<typename ...Args>
2 void print(Args... args) {
3   ((std::cout << args << " ") , ...);
4 }
```

VARIADIC TEMPLATES

C++17'S FOLD EXPRESSIONS

Syntax

```
(pattern op ...);
```

A fold repeats one expression for each element of the pack.

```
1 template<typename ...Args>
2 void print(Args... args) {
3   ((std::cout << args << " ") , ...);
4 }
```

Calling `print(1, 2.5, "hello")` prints `1 2.5 hello`

VARIADIC TEMPLATES

C++17'S FOLD EXPRESSIONS

Syntax

```
(pattern op ...);
```

A fold repeats one expression for each element of the pack.

```
1 template<typename ...Args>
2 void print(Args... args) {
3   ((std::cout << args << " ") , ...);
4 }
```

Calling `print(1, 2.5, "hello")` prints `1 2.5 hello`

ℹ️ Before C++17 you peeled the pack off one argument at a time, with a recursive overload and a non-template base case

VARIADIC TEMPLATES

UNDER THE HOOD: ONE CALL PER PACK ELEMENT 🔎

cppinsights.io/s/16760da2

Source:

```
1 template<typename ...Args>
2 void print(Args... args)
3 {
4   ((std::cout << args << " ") , ...);
5 }
6
7 print(1, 2.5, "hello");
```

Insight:

```
1  template<>
2  void print<int, double, const char *>(int __args0,
3                                        double __args1,
4                                        const char * __args2)
5  {
6    (std::cout << __args0 << " "),
7    (std::cout << __args1 << " "),
8    (std::cout << __args2 << " ");
9  }
10
11 print(1, 2.5, "hello");
```

https://cppinsights.io/s/16760da2

https://en.cppreference.com/cpp/language/fold

PERFECT FORWARDING

Passing arguments on to another function while preserving their value category (lvalue or rvalue).

```
1 template<typename Args>
2 void relay(Args&& args) {
3   consume(std::forward<Args>(args));
4 }
```

PERFECT FORWARDING

Passing arguments on to another function while preserving their value category (lvalue or rvalue).

```
1 template<typename Args>
2 void relay(Args&& args) {
3   consume(std::forward<Args>(args));
4 }
```

TEXTBOOK EXAMPLE: `std::vector::emplace_back`

```
1 template<typename ...Args>
2 reference emplace_back(Args&&... args);
```

`push_back` builds a temporary, then moves it

```
users.push_back(User{2, "Bob"});
```

`emplace_back` calls the constructor in place

```
users.emplace_back(2, "Bob");
```

https://godbolt.org/z/334GsGa4n

PERFECT FORWARDING

THE `std::forward` CATCH

`std::forward` only works on a **forwarding reference**: a `T&&` where `T` is deduced **by this very function**.

```
1 template <typename T>
2 void relay(T&& arg) {
3   consume(std::forward<T>(arg));
4 }
```

PERFECT FORWARDING

THE `std::forward` CATCH

`std::forward` only works on a **forwarding reference**: a `T&&` where `T` is deduced **by this very function**.

```
1 template <typename T>
2 void relay(T&& arg) {
3   consume(std::forward<T>(arg));
4 }
```

⚠️ These are **not** forwarding references ⚠️

```
1 void relay(std::vector<int>&& arg); // fixed type
2
3 template<typename T>
4 void relay(const T&& arg);          // const rvalue
```

PERFECT FORWARDING

THE `std::forward` CATCH

`std::forward` only works on a **forwarding reference**: a `T&&` where `T` is deduced **by this very function**.

```
1 template <typename T>
2 void relay(T&& arg) {
3   consume(std::forward<T>(arg));
4 }
```

⚠️ These are **not** forwarding references ⚠️

```
1 void relay(std::vector<int>&& arg); // fixed type
2
3 template<typename T>
4 void relay(const T&& arg);          // const rvalue
```

Two more traps:

- ⚠️ Using `arg` again after forwarding it → moved-from bug
- ⚠️ `std::move` instead of `std::forward` → moves lvalues unconditionally

PERFECT FORWARDING

THE `std::forward` CATCH

`std::forward` only works on a **forwarding reference**: a `T&&` where `T` is deduced **by this very function**.

```
1 template <typename T>
2 void relay(T&& arg) {
3   consume(std::forward<T>(arg));
4 }
```

⚠️ These are **not** forwarding references ⚠️

```
1 void relay(std::vector<int>&& arg); // fixed type
2
3 template<typename T>
4 void relay(const T&& arg);          // const rvalue
```

Two more traps:

- ⚠️ Using `arg` again after forwarding it → moved-from bug
- ⚠️ `std::move` instead of `std::forward` → moves lvalues unconditionally

Forwarding references are also called **universal references**: Scott Meyers

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

```
template<typename T>
class Stack {
private:
  std::vector<T> data;
public:
  void pop() { data.pop_back(); }
  const T& top() const { return data.back(); }
  bool empty() const { return data.empty(); }
  std::size_t size() const { return data.size(); }

  template <typename ...U>
  void emplace(U&&... args) { data.emplace_back(std::forward<U>(args)...); }
};

Stack<int> intStack;
```

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

```
template<typename T>
class Stack {
private:
  std::vector<T> data;
public:
  void pop() { data.pop_back(); }
  const T& top() const { return data.back(); }
  bool empty() const { return data.empty(); }
  std::size_t size() const { return data.size(); }

  template <typename ...U>
  void emplace(U&&... args) { data.emplace_back(std::forward<U>(args)...); }
};

Stack<int> intStack;
```

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

```
template<typename T>
class Stack {
private:
  std::vector<T> data;
public:
  void pop() { data.pop_back(); }
  const T& top() const { return data.back(); }
  bool empty() const { return data.empty(); }
  std::size_t size() const { return data.size(); }

  template <typename ...U>
  void emplace(U&&... args) { data.emplace_back(std::forward<U>(args)...); }
};

Stack<int> intStack;
```

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

```
template<typename T>
class Stack {
private:
  std::vector<T> data;
public:
  void pop() { data.pop_back(); }
  const T& top() const { return data.back(); }
  bool empty() const { return data.empty(); }
  std::size_t size() const { return data.size(); }

  template <typename ...U>
  void emplace(U&&... args) { data.emplace_back(std::forward<U>(args)...); }
};

Stack<int> intStack;
Stack<float> floatStack;
```

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

```
template <typename T>
class Stack {
private:
  std::vector<T> data;
public:
  void pop() { data.pop_back(); }
  const T& top() const { return data.back(); }
  bool empty() const { return data.empty(); }
  std::size_t size() const { return data.size(); }

  template <typename ...U>
  void emplace(U&&... args) { data.emplace_back(std::forward<U>(args)...); }
};

Stack<int> intStack;
Stack<float> floatStack;
```

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

```
template<typename T>
class Stack {
private:
  std::vector<T> data;
public:
  void pop() { data.pop_back(); }
  const T& top() const { return data.back(); }
  bool empty() const { return data.empty(); }
  std::size_t size() const { return data.size(); }

  template <typename ...U>
  void emplace(U&&... args) { data.emplace_back(std::forward<U>(args)...); }
};

Stack<int> intStack;
```

`Stack<int>` and `Stack<float>` are **different types** generated from the same template.

CLASS/STRUCT TEMPLATE

DEFINES A FAMILY OF CLASSES

A class template describes one class blueprint that the compiler instantiates for each type.

```
template<typename T>
class Stack {
private:
  std::vector<T> data;
public:
  void pop() { data.pop_back(); }
  const T& top() const { return data.back(); }
  bool empty() const { return data.empty(); }
  std::size_t size() const { return data.size(); }

  template <typename ...U>
  void emplace(U&&... args) { data.emplace_back(std::forward<U>(args)...); }
};

Stack<int> intStack;
```

`Stack<int>` and `Stack<float>` are **different types** generated from the same template.

Because `Stack<int>` and `Stack<float>` are types, they can be given as template parameter values

PERFECT FORWARDING

THE `std::forward` CATCH RELOADED

⚠️ Not all `T&&` are forwarding references

```
1 template <typename T> // T deduced HERE, at instantiation
2 class Stack {
3   // ... other member functions ...
4
5   // args looks like a forwarding reference but T is already fixed
6   void emplace(T&&... args) { data.emplace_back(std::forward<T>(args)...); }
7 };
```

PERFECT FORWARDING

THE `std::forward` CATCH RELOADED

⚠️ Not all `T&&` are forwarding references

```cpp
template <typename T> // T deduced HERE, at instantiation
class Stack {
  // ... other member functions ...

  // args looks like a forwarding reference but T is already fixed
  void emplace(T&&... args) { data.emplace_back(std::forward<T>(args)...); }
};
```

Example of instantiation: `Stack<int> s;`

```cpp
struct Stack {
  // ... other member functions ...

  // ⚠️ Only accept rvalues references of type int
  void emplace(int&&... args) { data.emplace_back(std::forward<int>(args)...); }
};
```

PERFECT FORWARDING

THE `std::forward` CATCH RELOADED

⚠ Not all T&&

```
template <typer
class Stack {
  // ... other
  // args looks
  void emplace(
};
```

Example of inst

```
struct Stack {
  // ... other member functions ...

  // ⚠ Only accept rvalues references of type int
  void emplace(int&&... args) { data.emplace_back(std::forward<int>(args)...); }
};
```

Fix: make `emplace` templated

```
template <typename T>
struct Stack {
  // ... other member functions ...

  template <typename ...U>
  void emplace(U&&... args) {
    consume(std::forward<U>(args)...);
  }
};
```

CLASS/STRUCT TEMPLATE

FUNCTION MEMBER INSTANTIATION

```
1 void useStack() {
2   Stack<int> s{};
3   s.emplace(5);
4   std::printf("%d\n", s.top());
5 }
```

https://godbolt.org/z/n7K1PKdrs

Only the called member functions are instantiated

CLASS/STRUCT TEMPLATE

FUNCTION MEMBER INSTANTIATION

```
1 void useStack() {
2   Stack<int> s{};
3   s.emplace(5);
4   std::printf("%d\n", s.top());
5 }
```

https://godbolt.org/z/n7K1PKdrs

Only the called member functions are instantiated

⚠ A member function that would not compile for `T` stays silent until someone calls it

CLASS/STRUCT TEMPLATE

CURIOUSLY RECURRING TEMPLATE PATTERN (CRTP)

A class can pass **itself** as the template argument when deriving from it.

CLASS/STRUCT TEMPLATE

CURIOUSLY RECURRING TEMPLATE PATTERN (CRTP)

A class can pass **itself** as the template argument when deriving from it.

ℹ️ The base class knows the type `CHILD` of the class deriving from it

```cpp
template <typename CHILD>
struct Base {
  void greet() {
    static_cast<CHILD*>(this)->greet_impl(); // Base knows the derived type
  }
};

struct Derived : public Base<Derived> { // pass yourself as the argument
  void greet_impl() { std::puts("hello"); }
};

Derived d;
d.greet(); // prints "hello", no virtual call
```

CLASS/STRUCT TEMPLATE

CURIOUSLY RECURRING TEMPLATE PATTERN (CRTP)

A class can pass **itself** as the template argument when deriving from it.

ℹ️ The base class knows the type `CHILD` of the class deriving from it

```cpp
template <typename CHILD>
struct Base {
  void greet() {
    static_cast<CHILD*>(this)->greet_impl(); // Base knows the derived type
  }
};

struct Derived : public Base<Derived> { // pass yourself as the argument
  void greet_impl() { std::puts("hello"); }
};

Derived d;
d.greet(); // prints "hello", no virtual call
```

CLASS/STRUCT TEMPLATE

CURIOUSLY RECURRING TEMPLATE PATTERN (CRTP)

A class can pass **itself** as the template argument when deriving from it.

ℹ️ The base class knows the type `CHILD` of the class deriving from it

```cpp
template <typename CHILD>
struct Base {
  void greet() {
    static_cast<CHILD*>(this)->greet_impl(); // Base knows the derived type
  }
};

struct Derived : public Base<Derived> { // pass yourself as the argument
  void greet_impl() { std::puts("hello"); }
};

Derived d;
d.greet(); // prints "hello", no virtual call
```

CLASS/STRUCT TEMPLATE

CURIOUSLY RECURRING TEMPLATE PATTERN (CRTP)

A class can pass **itself** as the template argument when deriving from it.

ℹ️ The base class knows the type `CHILD` of the class deriving from it

```cpp
template <typename CHILD>
struct Base {
  void greet() {
    static_cast<CHILD*>(this)->greet_impl(); // Base knows the derived type
  }
};

struct Derived : public Base<Derived> { // pass yourself as the argument
  void greet_impl() { std::puts("hello"); }
};

Derived d;
d.greet(); // prints "hello", no virtual call
```

CLASS/STRUCT TEMPLATE

CURIOUSLY RECURRING TEMPLATE PATTERN (CRTP)

A class can pass **itself** as the template argument when deriving from it.

ℹ️ The base class knows the type `CHILD` of the class deriving from it

```cpp
template <typename CHILD>
struct Base {
  void greet() {
    static_cast<CHILD*>(this)->greet_impl(); // Base knows the derived type
  }
};

struct Derived : public Base<Derived> { // pass yourself as the argument
  void greet_impl() { std::puts("hello"); }
};

Derived d;
d.greet(); // prints "hello", no virtual call
```

CLASS/STRUCT TEMPLATE

CURIOUSLY RECURRING TEMPLATE PATTERN (CRTP)

A class can pass **itself** as the template argument when deriving from it.

ℹ️ The base class knows the type `CHILD` of the class deriving from it

```cpp
template <typename CHILD>
struct Base {
  void greet() {
    static_cast<CHILD*>(this)->greet_impl(); // Base knows the derived type
  }
};

struct Derived : public Base<Derived> { // pass yourself as the argument
  void greet_impl() { std::puts("hello"); }
};

Derived d;
d.greet(); // prints "hello", no virtual call
```

💡 Static polymorphism, resolved at compile time 💡

LET'S SEE TEMPLATES IN ACTION



GENERIC SERIALIZER USING A POLICY-BASED DESIGN

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

- Design a `dump_object` function to serialize an object into different formats (e.g., JSON, YAML)

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

- Design a `dump_object` function to serialize an object into different formats (e.g., JSON, YAML)
- It should use policy classes (sinks) to determine the output format

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

- Design a `dump_object` function to serialize an object into different formats (e.g., JSON, YAML)
- It should use policy classes (sinks) to determine the output format
- No inheritance required

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

- Design a `dump_object` function to serialize an object into different formats (e.g., JSON, YAML)
- It should use policy classes (sinks) to determine the output format
- No inheritance required
- Easy to extend with new sinks without modifying the function itself

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

```
template <typename Sink>
void dump_object(const Object& d)
{
  Sink::begin_object("object");
  Sink::field("id", d.id());
  Sink::field("name", d.name());
  Sink::end_object();
}
```

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

```
template <typename Sink>
void dump_object(const Object& d)
{
  Sink::begin_object("object");
  Sink::field("id", d.id());
  Sink::field("name", d.name());
  Sink::end_object();
}
```

```
struct JsonSink {
  static void begin_object(std::string_view n) {
    std::printf("\"%s\": {\n", n.data());
  }
  static void field(std::string_view k, int v) {
    std::printf("  \"%s\": %d,\n", k.data(), v);
  }
  static void field(std::string_view k, std::string_view v) {
    std::printf("  \"%s\": \"%s\",\n", k.data(), v.data());
  }
  static void end_object() { std::printf("}\n"); }
};
```

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

```
template <typename Sink>
void dump_object(const Object& d)
{
  Sink::begin_object("object");
  Sink::field("id", d.id());
  Sink::field("name", d.name());
  Sink::end_object();
}
```

```
struct YamlSink {
  static void begin_object(std::string_view n) {
    std::printf("%s:\n", n.data());
  }
  static void field(std::string_view k, int v) {
    std::printf("  %s: %d\n", k.data(), v);
  }
  static void field(std::string_view k, std::string_view v) {
    std::printf("  %s: %s\n", k.data(), v.data());
  }
  static void end_object() {}
};
```

GENERIC SERIALIZER USING A POLICY-BASED DESIGN

```
1 template <typename Sink>
2 void dump_object(const Object& d)
3 {
4   Sink::begin_object("object");
5   Sink::field("id", d.id());
6   Sink::field("name", d.name());
7   Sink::end_object();
8 }
```

Main function

```
1 int main() {
2   Object apple{1, "apple"};
3   std::printf("---------JSON----------\n");
4   dump_object<JsonSink>(apple);
5   std::printf("---------YAML----------\n");
6   dump_object<YamlSink>(apple);
7 }
```

https://godbolt.org/z/crqP67bTW

Output:

```
---------JSON----------
"object": {
  "id": 1,
  "name": "apple",
}
---------YAML----------
object:
  id: 1
  name: "apple"
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

- Register bank is used to program the hardware

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

- Register bank is used to program the hardware
- 2 set of registers bank exists: Active and Standby

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

- Register bank is used to program the hardware
- 2 set of registers bank exists: Active and Standby
- Driver writes the standby bank using `write(...)`
  - `write` takes an unspecified number of parameters
  - In our case it is `write(name, value)`

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

- Register bank is used to program the hardware
- 2 set of registers bank exists: Active and Standby
- Driver writes the standby bank using `write(...)`
  - `write` takes an unspecified number of parameters
  - In our case it is `write(name, value)`
- Driver calls `commit()` to swap the active and standby banks

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

- Register bank is used to program the hardware
- 2 set of registers bank exists: Active and Standby
- Driver writes the standby bank using `write(...)`
  - `write` takes an unspecified number of parameters
  - In our case it is `write(name, value)`
- Driver calls `commit()` to swap the active and standby banks
- Bank-swap policy is decoupled from register access logic

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

- Register bank is used to program the hardware
- 2 set of registers bank exists: Active and Standby
- Driver writes the standby bank using `write(...)`
  - `write` takes an unspecified number of parameters
  - In our case it is `write(name, value)`
- Driver calls `commit()` to swap the active and standby banks
- Bank-swap policy is decoupled from register access logic
- The driver code is unaware of the active bank

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

Driver code

```cpp
int main()
{
  ActiveStandbyBank<MotorConfig> mgr(MotorConfig{});

  // Write to the current standby bank i.e bank 0
  if(!mgr.write("speed_rpm", 1500)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
  if(!mgr.write("torque_limit", 80)) {
    std::cerr << "Cannot write torque_limit\n";
  }

  mgr.commit();

  // Now writing to bank 1 (the new standby bank)
  if(!mgr.write("speed_rpm", 1000)) {
    std::cerr << "Cannot write speed_rpm\n";
  }
}
```

Register access logic implementation

```cpp
class MotorConfig
{
private:
  struct Config { uint16_t speed_rpm = 0;
                  uint16_t torque_limit = 0; };
  Config m_banks[2]{};
public:
  bool write(uint8_t bank_id, std::string_view field, uint16_t value)
  {
    if (field == "speed_rpm") {
      m_banks[bank_id].speed_rpm = value;
      return true;
    } else if (field == "torque_limit") {
      m_banks[bank_id].torque_limit = value;
      return true;
    }
    return false;
  }

  void commit(uint8_t bank_id)
  {
    // action to commit the bank
  }
};
```

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
template <typename REGISTER_ACCESS_LOGIC>
class ActiveStandbyBank
{
private:
  REGISTER_ACCESS_LOGIC m_register_access_logic{};
  uint8_t m_active_id = 0;
public:
  explicit ActiveStandbyBank(REGISTER_ACCESS_LOGIC register_access_logic)
    : m_register_access_logic(std::move(register_access_logic)){}

  uint8_t active_id() const  { return m_active_id; }
  uint8_t standby_id() const { return (m_active_id + 1) % 2; }

  template <typename ...Args>
  bool write(Args&&... args)
```

https://godbolt.org/z/zd5EG59P4

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
 1 template <typename REGISTER_ACCESS_LOGIC>
 2 class ActiveStandbyBank
 3 {
 4 private:
 5   REGISTER_ACCESS_LOGIC m_register_access_logic{};
 6   uint8_t m_active_id = 0;
 7 public:
 8   explicit ActiveStandbyBank(REGISTER_ACCESS_LOGIC register_access_logic)
 9     : m_register_access_logic(std::move(register_access_logic)){}
10
11   uint8_t active_id() const  { return m_active_id; }
12   uint8_t standby_id() const { return (m_active_id + 1) % 2; }
13
14   template <typename ...Args>
15   bool write(Args&&... args)
```

https://godbolt.org/z/zd5EG59P4

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
template <typename REGISTER_ACCESS_LOGIC>
class ActiveStandbyBank
{
private:
  REGISTER_ACCESS_LOGIC m_register_access_logic{};
  uint8_t m_active_id = 0;
public:
  explicit ActiveStandbyBank(REGISTER_ACCESS_LOGIC register_access_logic)
    : m_register_access_logic(std::move(register_access_logic)){}

  uint8_t active_id() const  { return m_active_id; }
  uint8_t standby_id() const { return (m_active_id + 1) % 2; }

  template <typename ...Args>
  bool write(Args&&... args)
```

https://godbolt.org/z/zd5EG59P4

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
class ActiveStandbyBank
{
private:
  REGISTER_ACCESS_LOGIC m_register_access_logic{};
  uint8_t m_active_id = 0;
public:
  explicit ActiveStandbyBank(REGISTER_ACCESS_LOGIC register_access_logic)
    : m_register_access_logic(std::move(register_access_logic)){}

  uint8_t active_id() const  { return m_active_id; }
  uint8_t standby_id() const { return (m_active_id + 1) % 2; }

  template <typename ...Args>
  bool write(Args&&... args)
  {
    return m_register_access_logic.write(standby_id(), std::forward<Args>(args)...);
  }
```

https://godbolt.org/z/zd5EG59P4

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
explicit ActiveStandbyBank(REGISTER_ACCESS_LOGIC register_access_logic)
  : m_register_access_logic(std::move(register_access_logic)){}

uint8_t active_id() const  { return m_active_id; }
uint8_t standby_id() const { return (m_active_id + 1) % 2; }

template <typename ...Args>
bool write(Args&&... args)
{
  return m_register_access_logic.write(standby_id(), std::forward<Args>(args)...);
}

template <typename ...Args>
void commit(Args&&... args)
{
  const auto current_standby_id = standby_id();
  m_active_id = current_standby_id;
```

https://godbolt.org/z/zd5EG59P4

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
 7 public:
 8   explicit ActiveStandbyBank(REGISTER_ACCESS_LOGIC register_access_logic)
 9     : m_register_access_logic(std::move(register_access_logic)){}
10
11   uint8_t active_id() const  { return m_active_id; }
12   uint8_t standby_id() const { return (m_active_id + 1) % 2; }
13
14   template <typename ...Args>
15   bool write(Args&&... args)
16   {
17     return m_register_access_logic.write(standby_id(), std::forward<Args>(args)...);
18   }
19
20   template <typename ...Args>
21   void commit(Args&&... args)
22   {
```

https://godbolt.org/z/zd5EG59P4

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
    : m_register_access_logic(std::move(register_access_logic)){}

  uint8_t active_id() const  { return m_active_id; }
  uint8_t standby_id() const { return (m_active_id + 1) % 2; }

  template <typename ...Args>
  bool write(Args&&... args)
  {
    return m_register_access_logic.write(standby_id(), std::forward<Args>(args)...);
  }

  template <typename ...Args>
  void commit(Args&&... args)
  {
    const auto current_standby_id = standby_id();
    m_active_id = current_standby_id;
    m_register_access_logic.commit(current_standby_id, std::forward<Args>(args)...);
```

https://godbolt.org/z/zd5EG59P4

GENERIC ABSTRACTION LAYER

REGISTER BANK MANAGER

ActiveStandbyBank implementation

```
13
14    template <typename ...Args>
15    bool write(Args&&... args)
16    {
17      return m_register_access_logic.write(standby_id(), std::forward<Args>(args)...);
18    }
19
20    template <typename ...Args>
21    void commit(Args&&... args)
22    {
23      const auto current_standby_id = standby_id();
24      m_active_id = current_standby_id;
25      m_register_access_logic.commit(current_standby_id, std::forward<Args>(args)...);
26    }
27  };
```

https://godbolt.org/z/zd5EG59P4

FLEXIBLE AND SAFE REGISTER ACCESSES

CONCEPT

- Hardware is programmed through registers
- Each register has fields of different sizes
- Modifying a field is a generic **read-modify-write** (RMW) algorithm
  Read the register, **clear** the field's bits, **set** the new value, write it back

For instance: ARM PL011 UART has a register called UARTLCR_H

Bits	Name
15:8	Reserved
7	SPS - Stick parity select
6:5	WLEN - Word length
4	FEN - Enable FIFOs

Bits	Name
3	STP2 - Two stop bits select
2	EPS - Even parity select
1	PEN - Parity enable
0	BRK - Send break

FLEXIBLE AND SAFE REGISTER ACCESSES

GOAL

- Provide a `Register::write(...)` function to write the fields
  - Multiple fields can be written at once i.e multiple parameters
  - We can choose how many field to write at once
  - The order of parameters doesn't matter
- Type safe

FLEXIBLE AND SAFE REGISTER ACCESSES

GOAL

- Provide a `Register::write(...)` function to write the fields
  - Multiple fields can be written at once i.e multiple parameters
  - We can choose how many field to write at once
  - The order of parameters doesn't matter
- Type safe

Client code

```
UARTLCR_H lcrh{registerAddress};

lcrh.write(
  // 8 data bits
  UARTLCR_H::WLEN{0b11},
  // enable FIFOs
  UARTLCR_H::FEN{0b1},
  // no parity
  UARTLCR_H::PEN{0b0}
);
```

FLEXIBLE AND SAFE REGISTER ACCESSES

GOAL

- Provide a `Register::write(...)` function to write the fields
  - Multiple fields can be written at once i.e multiple parameters
  - We can choose how many field to write at once
  - The order of parameters doesn't matter
- Type safe

Client code

```
UARTLCR_H lcrh{registerAddress};

lcrh.write(
  // 8 data bits
  UARTLCR_H::WLEN{0b11},
  // enable FIFOs
  UARTLCR_H::FEN{0b1},
  // no parity
  UARTLCR_H::PEN{0b0}
);
```

```
UARTLCR_H lcrh{registerAddress};

lcrh.write(
  // enable FIFOs
  UARTLCR_H::FEN{0b1},
  // no parity
  UARTLCR_H::PEN{0b0},
  // 8 data bits
  UARTLCR_H::WLEN{0b11}
);
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
1  // Use CRTP to give the parent class the type of the register it abstracts
2  struct UARTLCR_H : public Register<UARTLCR_H> {
3    UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}
4
5    struct WLEN {
6      using REG = UARTLCR_H;
7      static constexpr uint32_t FIELD_OFFSET = 5;
8      static constexpr uint32_t FIELD_SIZE = 2;
9      static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
10     uint32_t value;
11     WLEN(uint32_t value):value{value} {}
12   };
13
14   struct PEN {
15     using REG = UARTLCR_H;
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
1  // Use CRTP to give the parent class the type of the register it abstracts
2  struct UARTLCR_H : public Register<UARTLCR_H> {
3    UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}
4
5    struct WLEN {
6      using REG = UARTLCR_H;
7      static constexpr uint32_t FIELD_OFFSET = 5;
8      static constexpr uint32_t FIELD_SIZE = 2;
9      static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
10     uint32_t value;
11     WLEN(uint32_t value):value{value} {}
12   };
13
14   struct PEN {
15     using REG = UARTLCR_H;
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
1  // Use CRTP to give the parent class the type of the register it abstracts
2  struct UARTLCR_H : public Register<UARTLCR_H> {
3    UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}
4
5    struct WLEN {
6      using REG = UARTLCR_H;
7      static constexpr uint32_t FIELD_OFFSET = 5;
8      static constexpr uint32_t FIELD_SIZE = 2;
9      static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
10     uint32_t value;
11     WLEN(uint32_t value):value{value} {}
12   };
13
14   struct PEN {
15     using REG = UARTLCR_H;
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
1  // Use CRTP to give the parent class the type of the register it abstracts
2  struct UARTLCR_H : public Register<UARTLCR_H> {
3    UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}
4
5    struct WLEN {
6      using REG = UARTLCR_H;
7      static constexpr uint32_t FIELD_OFFSET = 5;
8      static constexpr uint32_t FIELD_SIZE = 2;
9      static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
10     uint32_t value;
11     WLEN(uint32_t value):value{value} {}
12   };
13
14   struct PEN {
15     using REG = UARTLCR_H;
16     static constexpr uint32_t FIELD_OFFSET = 1;
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
1  // Use CRTP to give the parent class the type of the register it abstracts
2  struct UARTLCR_H : public Register<UARTLCR_H> {
3    UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}
4
5    struct WLEN {
6      using REG = UARTLCR_H;
7      static constexpr uint32_t FIELD_OFFSET = 5;
8      static constexpr uint32_t FIELD_SIZE = 2;
9      static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
10     uint32_t value;
11     WLEN(uint32_t value):value{value} {}
12   };
13
14   struct PEN {
15     using REG = UARTLCR_H;
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
// Use CRTP to give the parent class the type of the register it abstracts
struct UARTLCR_H : public Register<UARTLCR_H> {
  UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}

  struct WLEN {
    using REG = UARTLCR_H;
    static constexpr uint32_t FIELD_OFFSET = 5;
    static constexpr uint32_t FIELD_SIZE = 2;
    static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
    uint32_t value;
    WLEN(uint32_t value):value{value} {}
  };

  struct PEN {
    using REG = UARTLCR_H;
    static constexpr uint32_t FIELD_OFFSET = 1;
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
 3   UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}
 4
 5   struct WLEN {
 6     using REG = UARTLCR_H;
 7     static constexpr uint32_t FIELD_OFFSET = 5;
 8     static constexpr uint32_t FIELD_SIZE = 2;
 9     static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
10     uint32_t value;
11     WLEN(uint32_t value):value{value} {}
12   };
13
14   struct PEN {
15     using REG = UARTLCR_H;
16     static constexpr uint32_t FIELD_OFFSET = 1;
17     static constexpr uint32_t FIELD_SIZE = 1;
18     static constexpr uint32_t FIELD_MASK = ((1u << FIELD_SIZE) - 1) << FIELD_OFFSET;
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
    uint32_t value;
    WLEN(uint32_t value):value{value} {}
  };

  struct PEN {
    using REG = UARTLCR_H;
    static constexpr uint32_t FIELD_OFFSET = 1;
    static constexpr uint32_t FIELD_SIZE = 1;
    static constexpr uint32_t FIELD_MASK = ((1u << FIELD_SIZE) - 1) << FIELD_OFFSET;
    uint32_t value;
    PEN(uint32_t value):value{value} {}
  };

    // Other fields implementation...
};
```

FLEXIBLE AND SAFE REGISTER ACCESSES

UARTLCR_H REGISTER DEFINITION

```
1  // Use CRTP to give the parent class the type of the register it abstracts
2  struct UARTLCR_H : public Register<UARTLCR_H> {
3    UARTLCR_H(volatile uint32_t* regAddr):Register{regAddr} {}
4
5    struct WLEN {
6      using REG = UARTLCR_H;
7      static constexpr uint32_t FIELD_OFFSET = 5;
8      static constexpr uint32_t FIELD_SIZE = 2;
9      static constexpr uint32_t FIELD_MASK = ((1 << FIELD_SIZE) - 1) << FIELD_OFFSET;
10     uint32_t value;
11     WLEN(uint32_t value):value{value} {}
12   };
13
14   struct PEN {
15     using REG = UARTLCR_H;
```

ℹ️ Each sub-structure implements the implicit requirements of the register abstraction

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
template<typename CHILD>
class Register{
private:
  volatile uint32_t* value; // Points to the actual memory-mapped register
public:
  Register(volatile uint32_t* regAddr):value{regAddr}
  {}

  uint32_t& operator*() {
    return *value;
  }

  template <typename ...FIELD>
  void write(FIELD... field)
  {
    static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
                  "Using field from the wrong register");
    uint32_t curVal = *value; // Read value

    auto applyField = [&](auto f)
    {
      using FIELD_TYPE = decltype(f);
      curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
```

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
template<typename CHILD>
class Register{
private:
  volatile uint32_t* value; // Points to the actual memory-mapped register
public:
  Register(volatile uint32_t* regAddr):value{regAddr}
  {}

  uint32_t& operator*() {
    return *value;
  }

  template <typename ...FIELD>
  void write(FIELD... field)
  {
    static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
                  "Using field from the wrong register");
    uint32_t curVal = *value; // Read value

    auto applyField = [&](auto f)
    {
      using FIELD_TYPE = decltype(f);
      curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
```

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
template<typename CHILD>
class Register{
private:
  volatile uint32_t* value; // Points to the actual memory-mapped register
public:
  Register(volatile uint32_t* regAddr):value{regAddr}
  {}

  uint32_t& operator*() {
    return *value;
  }

  template <typename ...FIELD>
  void write(FIELD... field)
  {
    static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
                  "Using field from the wrong register");
    uint32_t curVal = *value; // Read value

    auto applyField = [&](auto f)
    {
      using FIELD_TYPE = decltype(f);
      curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
```

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
template<typename CHILD>
class Register{
private:
  volatile uint32_t* value; // Points to the actual memory-mapped register
public:
  Register(volatile uint32_t* regAddr):value{regAddr}
  {}

  uint32_t& operator*() {
    return *value;
  }

  template <typename ...FIELD>
  void write(FIELD... field)
  {
    static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
                  "Using field from the wrong register");
    uint32_t curVal = *value; // Read value

    auto applyField = [&](auto f)
    {
      using FIELD_TYPE = decltype(f);
      curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
```

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
 8
 9   uint32_t& operator*() {
10     return *value;
11   }
12
13   template <typename ...FIELD>
14   void write(FIELD... field)
15   {
16     static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
17                   "Using field from the wrong register");
18     uint32_t curVal = *value; // Read value
19
20     auto applyField = [&](auto f)
21     {
22       using FIELD_TYPE = decltype(f);
23       curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
24       curVal |= (f.value << FIELD_TYPE::FIELD_OFFSET) & FIELD_TYPE::FIELD_MASK; // Set the field's value
25     };
26     (applyField(field), ...);
27
28     *value = curVal; // Write back value
29   }
30 };
```

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
 2 class Register{
 3 private:
 4   volatile uint32_t* value; // Points to the actual memory-mapped register
 5 public:
 6   Register(volatile uint32_t* regAddr):value{regAddr}
 7   {}
 8
 9   uint32_t& operator*() {
10     return *value;
11   }
12
13   template <typename ...FIELD>
14   void write(FIELD... field)
15   {
16     static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
17                   "Using field from the wrong register");
18     uint32_t curVal = *value; // Read value
19
20     auto applyField = [&](auto f)
21     {
22       using FIELD_TYPE = decltype(f);
23       curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
24       curVal |= (f.value << FIELD_TYPE::FIELD_OFFSET) & FIELD_TYPE::FIELD_MASK; // Set the field's value
25     };
```

👍 Any number of fields

👍 Fields order doesn't matter

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
 5 public:
 6   Register(volatile uint32_t* regAddr):value{regAddr}
 7   {}
 8
 9   uint32_t& operator*() {
10     return *value;
11   }
12
13   template <typename ...FIELD>
14   void write(FIELD... field)
15   {
16     static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
17                   "Using field from the wrong register");
18     uint32_t curVal = *value; // Read value
19
20     auto applyField = [&](auto f)
21     {
22       using FIELD_TYPE = decltype(f);
23       curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
24       curVal |= (f.value << FIELD_TYPE::FIELD_OFFSET) & FIELD_TYPE::FIELD_MASK; // Set the field's value
25     };
26     (applyField(field), ...);
27
28     *value = curVal; // Write back value
```

💡 `static_assert` + `<type_traits>` ensure fields from the right register are used 💡

FLEXIBLE AND SAFE REGISTER ACCESSES

REGISTER ABSTRACTION IMPLEMENTATION

```
 8
 9   uint32_t& operator*() {
10     return *value;
11   }
12
13   template <typename ...FIELD>
14   void write(FIELD... field)
15   {
16     static_assert((std::is_same_v<typename FIELD::REG, CHILD> && ...),
17                   "Using field from the wrong register");
18     uint32_t curVal = *value; // Read value
19
20     auto applyField = [&](auto f)
21     {
22       using FIELD_TYPE = decltype(f);
23       curVal &= ~FIELD_TYPE::FIELD_MASK;                                    // Clear the field
24       curVal |= (f.value << FIELD_TYPE::FIELD_OFFSET) & FIELD_TYPE::FIELD_MASK; // Set the field's value
25     };
26     (applyField(field), ...);
27
28     *value = curVal; // Write back value
29   }
30 };
```

ℹ️ The fold expression executes the lambda for each field

FLEXIBLE AND SAFE REGISTER ACCESSES

WHAT DOES ALL OF THIS COST?

The abstraction executes a lambda for each field...

FLEXIBLE AND SAFE REGISTER ACCESSES

WHAT DOES ALL OF THIS COST?

The abstraction executes a lambda for each field...

```
uint32_t testValueAbstraction() {
  uint32_t v{0};
  UARTLCR_H lcrh{&v};
  lcrh.write(
    UARTLCR_H::WLEN{0b11},
    UARTLCR_H::FEN{0b1},
    UARTLCR_H::PEN{0b0}
  );
  return *lcrh;
}
```

```
uint32_t testValueDirect() {
  volatile uint32_t v{0};

  v = (v & ~0x72) | 0x70;

  return v;
}
```

Is the abstraction slower, faster or the same?



FLEXIBLE AND SAFE REGISTER ACCESSES

TRUE ZERO COST ABSTRACTION

```
uint32_t testValueAbstraction() {
  uint32_t v{0};
  UARTLCR_H lcrh{&v};
  lcrh.write(
    UARTLCR_H::WLEN{0b11},
    UARTLCR_H::FEN{0b1},
    UARTLCR_H::PEN{0b0}
  );
  return *lcrh;
}
```

```
uint32_t testValueDirect() {
  volatile uint32_t v{0};

  v = (v & ~0x72) | 0x70;

  return v;
}
```

```
testValueAbstraction():
    sub     sp, sp, #16
    str     wzr, [sp, 12]
    ldr     w0, [sp, 12]
    and     w0, w0, -3
    orr     w0, w0, 112
    str     w0, [sp, 12]
    ldr     w0, [sp, 12]
    add     sp, sp, 16
    ret
```

```
testValueDirect():
    sub     sp, sp, #16
    str     wzr, [sp, 12]
    ldr     w0, [sp, 12]
    and     w0, w0, -3
    orr     w0, w0, 112
    str     w0, [sp, 12]
    ldr     w0, [sp, 12]
    add     sp, sp, 16
    ret
```

https://godbolt.org/z/dfKK4nKoh

CONCLUSION: KEY TAKEAWAYS

CONCLUSION: KEY TAKEAWAYS

- 🔑 Template is a very powerful feature of C++

CONCLUSION: KEY TAKEAWAYS

- 🔑 Template is a very powerful feature of C++
- 🔑😈 Beware of the implicit requirements 😈

CONCLUSION: KEY TAKEAWAYS

- 🔑 Template is a very powerful feature of C++
- 🔑😈 Beware of the implicit requirements 😈
- 🔑 Replace your macros with templates

CONCLUSION: KEY TAKEAWAYS

- 🔑 Template is a very powerful feature of C++
- 🔑😈 Beware of the implicit requirements 😈
- 🔑 Replace your macros with templates
- 🔑 Generic programming

CONCLUSION: KEY TAKEAWAYS

- 🔑 Template is a very powerful feature of C++
- 🔑😈 Beware of the implicit requirements 😈
- 🔑 Replace your macros with templates
- 🔑 Generic programming
- 🔑 Allows extensibility

CONCLUSION: KEY TAKEAWAYS

- 🔑 Template is a very powerful feature of C++
- 🔑😈 Beware of the implicit requirements 😈
- 🔑 Replace your macros with templates
- 🔑 Generic programming
- 🔑 Allows extensibility
- 🔑 Good template abstraction can simplify client code without compromise on performance

CONCLUSION: KEY TAKEAWAYS

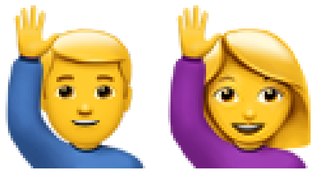
- 🔑 Template is a very powerful feature of C++
- 🔑😈 Beware of the implicit requirements 😈
- 🔑 Replace your macros with templates
- 🔑 Generic programming
- 🔑 Allows extensibility
- 🔑 Good template abstraction can simplify client code without compromise on performance

Don't be scared of templates 👻

THE END!



QUESTIONS?



Slides available at: laurentcarlier.com/b2b-template-cppcon-2026/